



Trabalho Final

Protocolo de transporte confiável sobre UDP

Objetivo

Este trabalho tem como objetivo o projeto, implementação e análise de um protocolo de transporte confiável sobre UDP. O protocolo, denominado **SRTP (Simple Reliable Transport Protocol)**, deve ser capaz de transferir arquivos de forma confiável entre dois hosts distintos em uma rede local, implementando mecanismos de controle de erros e de fluxo progressivamente mais sofisticados.

O trabalho é dividido em duas partes. A Parte 1 cobre as regras gerais do protocolo e a implementação stop-and-wait. A Parte 2 adiciona as implementações com janela deslizante (GBN e SR) e exige análise comparativa entre as três variantes.

A entrega final consiste em um **relatório técnico** e no **código-fonte** das implementações. Adicionalmente, cada grupo participará de um **teste de interoperabilidade** com outro grupo durante a aula de apresentação.

Descrição

Especificação do Protocolo SRTP

Camada de transporte subjacente

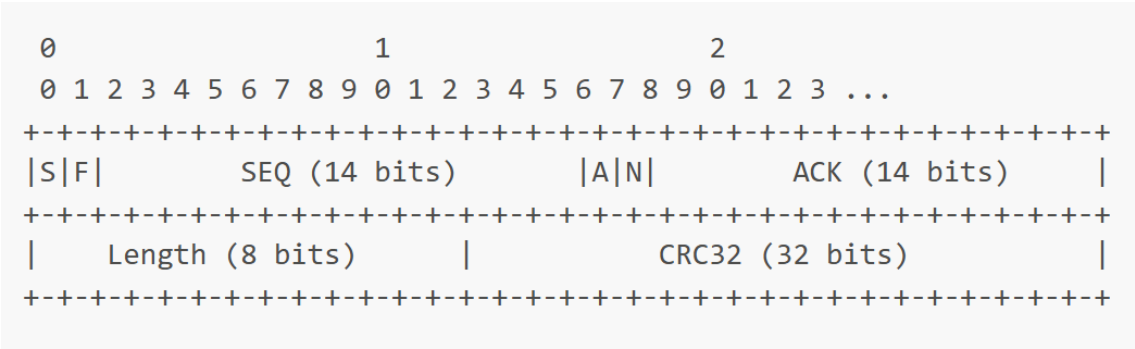
O SRTP opera sobre UDP. Toda confiabilidade, controle de fluxo e ordenação são responsabilidade do próprio protocolo, não do sistema operacional.

Modelo de comunicação

O SRTP é half-duplex: em um dado momento, apenas um lado transmite dados. O lado oposto envia exclusivamente pacotes de controle (ACK, NACK). Não há transmissão simultânea de dados nos dois sentidos.

Formato do cabeçalho

O cabeçalho SRTP possui 9 bytes, com o seguinte layout:



Campo	Bits	Descrição
SEQ	14	Número de sequência do pacote (em pacotes, não em bytes)
SYN	1	Flag de estabelecimento de conexão
FIN	1	Flag de encerramento de conexão
ACK	14	Número de acknowledgement
ACK flag	1	Indica que o campo ACK é válido
NACK	1	Negative acknowledgement
Length	8	Tamanho do payload em bytes; durante o handshake, tamanho de janela proposto
CRC32	32	Checksum CRC32 calculado sobre cabeçalho + payload

Semântica do campo Length

- Durante a transferência: número de bytes de payload neste pacote.

- Length = 255 → pacote intermediário; o receiver bufferiza e aguarda.
 - Length < 255 → último pacote do stream; o receiver entrega o buffer completo à aplicação (*push*).
 - Length = 0 → edge case: o arquivo é múltiplo exato de 255 bytes; sinaliza fim de stream sem payload residual.
- Durante o handshake (pacotes SYN e SYN+ACK): tamanho de janela proposto pelo lado que envia.

Número de sequência

- Contado em pacotes, não em bytes.
- Inicia em 0 no primeiro pacote de dados após o handshake.
- Espaço de sequência: 14 bits → valores de 0 a 16383. Wrap-around deve ser tratado corretamente.
- Todos os pacotes de dados têm payload de 255 bytes, exceto o último pacote do stream.

Checksum

- Algoritmo: CRC32.
- Calculado sobre o cabeçalho completo (com o campo CRC32 zerado) concatenado com o payload.
- Pacotes com CRC32 inválido devem ser descartados pelo receiver, sem envio de NACK. Como o pacote está corrompido, nenhum campo do cabeçalho é confiável, incluindo o SEQ. Sem um SEQ válido, não é possível construir um NACK significativo. O timeout do sender é responsável por disparar a retransmissão.

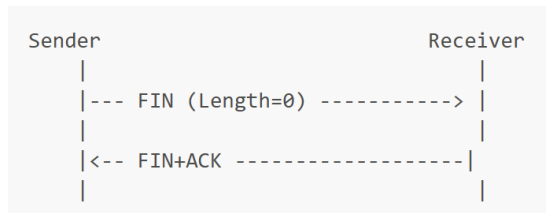
Estabelecimento de conexão (three-way handshake)



- O iniciador propõe sua janela no campo Length do SYN.

- O receiver responde com sua própria janela no SYN+ACK.
- A janela efetiva da sessão é o menor dos dois valores.
- O terceiro passo (ACK) confirma o estabelecimento antes do início da transferência.
- SEQ e ACK são 0 durante o handshake.

Encerramento de conexão (two-way)



- Iniciado pelo lado que transmitiu os dados, após o último pacote ser confirmado.
- O receiver confirma com FIN+ACK e encerra a sessão.

Timeout e retransmissão

- Timeout fixo: 100ms.
- Em caso de timeout sem ACK, o pacote (ou janela, dependendo da variante) é retransmitido.

Modelo de endereçamento e portas

O SRTP utiliza um par de portas simétricas derivado de um único parâmetro P:

- Ambos os lados recebem apenas P como argumento de porta.
- O lado em modo listen (receiver) escuta na porta P.
- O lado em modo connect (sender) conecta ao receiver na porta P e, somente após o handshake ser concluído, passa a escutar na porta P+1 para receber ACKs e NACKs.
- O modo de operação é definido pela presença ou ausência do argumento --listen.

Exemplo de uso:

```

# Receiver em modo listen na porta 6000
<executável> --listen --port 6000

# Sender conectando ao receiver (escuta ACKs em 6001, envia para 6000)
<executável> --host 192.168.1.10 --port 6000 --file arquivo.bin
  
```

Transferência bidirecional

O SRTP é half-duplex por sessão, mas suporta transferência nas duas direções através de sessões alternadas:

- O iniciador do handshake (quem envia o SYN) é sempre o sender de dados da sessão.
- Para transferir dados na direção oposta, o receiver da sessão anterior inicia uma nova conexão como sender.
- Cada direção exige um handshake e um encerramento independentes.

Parte 1: Stop-and-Wait

Regras de implementação

A primeira implementação do SRTP deve seguir o mecanismo stop-and-wait:

- O sender transmite um pacote por vez.
- O sender aguarda o ACK do receiver antes de transmitir o próximo pacote.
- Se o ACK não chegar dentro de 100ms, o pacote é retransmitido.
- O receiver deve enviar ACK para cada pacote recebido corretamente (CRC32 válido, SEQ esperado).
- Pacotes com CRC32 inválido são descartados silenciosamente, sem envio de NACK. Ver seção Checksum.
- Pacotes fora de ordem são descartados silenciosamente.

Testes obrigatórios

Todos os testes devem ser realizados entre máquinas distintas do grupo e acompanhados de capturas Wireshark (.pcapng).

Para cada cenário, deve ser transferido um arquivo de tamanho suficiente para gerar no mínimo 50 pacotes de dados.

Cenários de latência (sem perda, clumsy no sender ou receiver):

Cenário	Latência adicionada
L0	0ms (baseline)
L1	50ms

Cenário	Latência adicionada
L2	100ms
L3	150ms

Cenários de perda (sem latência adicional, clumsy no sender ou receiver):

Cenário	Taxa de perda
P0	0% (baseline)
P1	1%
P2	5%
P3	10%
P4	25%

O que deve aparecer no relatório

1. **Análise do throughput teórico máximo:** calcular a taxa máxima de transferência do stop-and-wait em função do RTT e do tamanho do pacote usando a fórmula de eficiência de protocolos ARQ (ver referência: Seção 3.4 de Kurose & Ross, *Redes de Computadores e a Internet: Uma abordagem top-down*). Confrontar o valor teórico com o throughput medido no cenário L0/P0.
2. **Análise dos cenários de latência:** para cada cenário L0–L3, reportar o throughput medido e o número de retransmissões. Explicar o comportamento observado em função do timeout de 100ms, especialmente a transição entre L1 e L2, e entre L2 e L3. As capturas Wireshark devem ser usadas como evidência.
3. **Análise dos cenários de perda:** para cada cenário P0–P4, reportar o throughput medido e o número de retransmissões. Explicar como a taxa de perda afeta o throughput e relacionar com a fórmula teórica de eficiência sob perda.
4. **Análise do comportamento do CRC32:** demonstrar com uma captura que um pacote corrompido é detectado e descartado. Explicar o que aconteceria se o CRC32 não estivesse presente.

Parte 2: Go-Back-N e Selective Repeat

Regras adicionais

A Parte 2 exige duas novas implementações do SRTP com janela deslizante: Go-Back-N (GBN) e Selective Repeat (SR). Ambas devem reutilizar o mesmo cabeçalho e as mesmas regras de handshake e encerramento definidas na Parte 1.

O modo de operação do protocolo, stop-and-wait, GBN ou SR, deve ser selecionado por um argumento de linha de comando, por exemplo `--mode [saw|gbn|sr]`. O mesmo binário deve suportar os três modos.

Go-Back-N:

- O sender mantém uma janela de até N pacotes em trânsito simultaneamente.
- O receiver aceita apenas pacotes na ordem esperada; pacotes fora de ordem são descartados silenciosamente.
- Em caso de timeout ou NACK, o sender retransmite todos os pacotes a partir do não confirmado.
- ACKs são cumulativos: ACK(n) confirma todos os pacotes até n inclusive.
- O campo NACK deve ser usado: o receiver envia NACK com o número do pacote esperado ao receber um pacote fora de ordem.

Selective Repeat:

- O sender mantém uma janela de até N pacotes em trânsito simultaneamente.
- O receiver aceita e bufferiza pacotes fora de ordem dentro da janela.
- Em caso de timeout ou NACK, o sender retransmite apenas o pacote não confirmado.
- ACKs são individuais: ACK(n) confirma apenas o pacote n.
- O campo NACK deve ser usado ao detectar uma lacuna: a flag NACK é ativada e o campo ACK carrega o número de sequência do pacote faltante.

Tamanho máximo de janela (GBN e SR): limitado a 255 pacotes, definido pelo campo Length de 8 bits negociado no handshake.

Testes obrigatórios

Os cenários de latência e perda da Parte 1 devem ser repetidos para GBN e SR, usando os mesmos parâmetros. Adicionalmente, ambas as implementações devem ser testadas com os seguintes cenários de reordenação (sem latência adicional, sem perda):

Cenário	Taxa de reordenação (clumsy)
R0	0% (baseline)
R1	10%
R2	25%

Para cada implementação, deve ser testado com pelo menos dois tamanhos de janela distintos (sugestão: 4 e 16 pacotes).

O que deve aparecer no relatório

1. **Análise comparativa sob latência:** comparar o throughput de stop-and-wait, GBN e SR nos cenários L0–L3. Explicar por que a janela deslizante muda o impacto da latência no throughput.
2. **Análise comparativa sob perda:** comparar o número de retransmissões e o throughput de GBN e SR nos cenários P1–P4. Explicar a diferença de comportamento entre os dois protocolos sob perda e relacionar com o mecanismo de retransmissão de cada um.
3. **Análise comparativa sob reordenação:** este é o ponto central da Parte 2. Para cada cenário R0–R2, descrever e comparar o comportamento de GBN e SR. As capturas Wireshark devem mostrar claramente: no GBN, o descarte e a retransmissão em lote após pacotes fora de ordem; no SR, a bufferização e a entrega em ordem após preenchimento de lacunas. Explicar por que os dois protocolos divergem neste cenário.
4. **Análise do impacto do tamanho de janela:** para cada protocolo, comparar os resultados obtidos com os dois tamanhos de janela testados. Discutir o trade-off entre janela maior (maior throughput potencial) e janela maior (maior custo de retransmissão no GBN sob perda).
5. **Conclusão comparativa:** síntese das três implementações: em quais condições de rede cada uma é mais adequada e por quê. A conclusão deve ser fundamentada nos dados medidos, não apenas na teoria.

Teste de Interoperabilidade

Durante a apresentação, cada grupo realizará um teste de transmissão com outro grupo, usando as implementações de ambos os grupos. O teste consiste em:

- Sender de um grupo conectando ao receiver do outro grupo, e vice-versa.
- Transferência de um arquivo.

- O teste é considerado bem-sucedido se o arquivo transferido for idêntico ao original (verificado por hash).

O teste de interoperabilidade avalia a conformidade da implementação com a especificação do protocolo.

Uso de Inteligência Artificial

O uso de ferramentas de Inteligência Artificial (IA) generativa é permitido durante todas as etapas deste trabalho, incluindo pesquisa, projeto, implementação, depuração, análise de resultados e redação do relatório.

O relatório deve conter uma seção intitulada "**Uso de Inteligência Artificial**", descrevendo de forma resumida:

- As ferramentas utilizadas (ex.: ChatGPT, Claude, Gemini, Copilot, Cursor, entre outras);
- Os modelos utilizados, quando conhecidos;
- Como as ferramentas foram empregadas durante o desenvolvimento do trabalho;
- As principais contribuições fornecidas pelas ferramentas de IA;
- Outras fontes de referência utilizadas, tais como livros, RFCs, artigos, documentações técnicas, sites e vídeos.

O relatório deve incluir alguns exemplos representativos de consultas realizadas a ferramentas de IA que tenham contribuído significativamente para o projeto, implementação ou análise dos resultados.

Importante: O uso de IA não reduz a nota do trabalho e não constitui infração acadêmica. Entretanto, todos os integrantes do grupo devem compreender integralmente a solução entregue e ser capazes de explicar, durante a apresentação e arguição:

- o funcionamento do protocolo implementado;
- as decisões de projeto adotadas;
- os mecanismos de controle de erros e fluxo implementados;
- os experimentos realizados;
- os resultados apresentados no relatório.

O professor poderá questionar qualquer trecho do código-fonte, do relatório ou das capturas apresentadas. A incapacidade de demonstrar compreensão

adequada da solução entregue poderá resultar em redução da nota ou atribuição de nota zero ao trabalho.

A autoria intelectual do trabalho permanece sendo responsabilidade integral dos alunos, independentemente das ferramentas utilizadas durante seu desenvolvimento.

Resultado e Entrega

Regras de Entrega

- **Grupo:** grupos de até 3 alunos.
- **Data de entrega: 28/06 às 23:59** via Moodle (véspera da apresentação)
- **Formato:** arquivo .zip contendo:
 - Código-fonte da implementação (sem binários, arquivos temporários ou de build).
 - Arquivo README com instruções de compilação/execução e descrição dos argumentos de linha de comando.
 - Diretório capturas/ com os arquivos .pcapng de **todos os cenários obrigatórios**, nomeados de acordo com o cenário (ex: saw_L2.pcapng, sr_R2_janela16.pcapng).
 - Relatório em PDF, documento único cobrindo Parte 1 e Parte 2 (até 5 páginas).
- **Apresentação:** 29/06 e 06/07
- **Observações Gerais:** É importante que **todos os integrantes dos grupos estejam aptos a apresentarem o trabalho** a partir do início da aula. Para a entrega, é esperado que apenas um dos integrantes envie pelo Moodle, até a data e hora especificadas, um **arquivo .zip com os nomes dos integrantes**.
- **IMPORTANTE:** Não serão aceitos trabalhos entregues fora do prazo. Trabalhos que não compilam ou que não executam não serão avaliados. Todos os trabalhos serão analisados e comparados. Caso seja identificada cópia de trabalhos, todos os trabalhos envolvidos receberão nota Zero. O uso de código gerado por LLM sem compreensão demonstrada na apresentação resultará em nota zero para o grupo.

Avaliação

Item	Peso
Parte 1 — Implementação stop-and-wait	20%
Conformidade com a especificação do protocolo (cabeçalho, handshake, encerramento, CRC32)	10%
Funcionamento correto da transferência (sem e com clumsy)	10%
Parte 1 — Relatório	20%
Análise do throughput teórico vs medido	7%
Análise dos cenários de latência com evidência em capturas	7%
Análise dos cenários de perda com evidência em capturas	6%
Parte 2 — Implementação GBN e SR	20%
Conformidade com a especificação (GBN e SR)	10%
Funcionamento correto da transferência (sem e com clumsy)	10%
Parte 2 — Relatório	30%
Análise do throughput teórico vs medido para GBN e SR	5%
Análise comparativa sob latência e perda	8%
Análise comparativa sob reordenação (com evidência em capturas)	10%
Conclusão comparativa fundamentada em dados	7%
Teste de interoperabilidade ao vivo	10%

Item	Peso
Sender do grupo funciona com receiver de outro grupo	5%
Receiver do grupo funciona com sender de outro grupo	5%